LLM エージェントの閾値型要約における再帰置換と差分保持の比較

著者未設定

Comparing Recursive Replacement and Delta Retention under Threshold-Triggered Summarization for LLM Agents

Author DETAILS PENDING,

DOI:10.14923/transj.???????????

あらまし LLM エージェントの同一閾値下で再帰置換と差分保持を既知 4 課題で比較した．成功は 3 件から 2 件へ減り，差分が届いた共通成功課題も増費用となった．使用量不明 1 呼出しを残し，品質を保つ総費用削減は確認できなかった．

キーワード LLM エージェント，文脈圧縮，差分要約，閾値制御，費用評価

1. ま え が き

大規模言語モデル（LLM）によるエージェントは，推論と行動を繰り返し [1]，生成したコードで外部環境を操作する [2]．長い課題では，蓄積した行動と観測を後続の入力に繰り返し含める費用が増える．履歴要約はこの入力を短くするが，要約の作成自体にも入力・出力費用がかかる．更に，課題の途中で必要な情報を失うと，再取得や誤った操作につながり得る．

本稿では，要約を再び要約するときの保持表現に着目する．旧要約と新しい履歴をまとめ直す再帰置換は，重複や古い記述を整理できる一方，既に記録した状態もモデルが書き直す．旧要約を固定して新情報を追加すれば，旧要約全体の再出力を要求せずに済むが，古い状態や冗長な記述が後続入力に残る．このトレードオフが，課題の最終成功と要約費を含む総費用にどう現れるかを調べる．

プロンプト圧縮 [3]，長期エージェントの圧縮方針の最適化 [4]，キャッシュを考慮した文脈管理 [5] を踏まえ，本比較では通常の入力長閾値と実行予算を共通にし，第 2 要約の処置を変える．比較対象は，再帰置換と，**旧要約保持・明示的な差分 *object* 出力・ホストによる結合**の組である．既知 4 課題の全実行を対象に，成功を失わずに全呼出しの論理入出力を減らせるかを評価する．結果は成功喪失と使用量不明を含み，この目標の達成を支持しなかった．

2. 共通閾値下の保持表現

2.1 ハーネスと比較方式

図 1 に構成を示す．actor は課題実行，focus は履歴要約を担当する LLM 呼出しであり，同じモデルを用いる．ホストは完全な履歴を保存し，actor にはシステム指示，課題，採用済み要約，要約位置より後の履歴を提示する．直近 2 ターンは要約対象から除外する．

TR（threshold recursive）は通常の再帰要約，TDO（threshold delta object）は旧要約を保持する差分要約である．両方式とも actor 入力が 8,192 トークン以上，前回機会以後の新規完了ターンが 3 以上で要約機会を設ける．機会は未送信・不採用を含めて最大 2 回とする．初回の入力構成と出力契約は共通であり，第 1 要約を採用できた場合だけ，第 2 機会の処置を分ける．

旧要約を S_1，新たな対象履歴を H_2 とする．両方式とも S_1 と H_2 を focus へ提示する．TR はこれらをまとめた要約 S_2 を返す．TDO は S_1 を参照しながら，新情報を記述した単一の差分 object D_2 だけを返す．ホストが採用した後の要約部分は，

$$\mathrm{TR} : S_2, \qquad \mathrm{TDO} : (S_1, D_2) \tag{1}$$

となる．TDO の S_1 は保存済み文字列をそのまま用い，モデルに旧要約の再出力を要求しない．初回が不採用なら両方式とも通常の要約経路を用いる．

2.2 共通の採用制約

要約 object は完了事項，利用可能状態，残作業，不確実性の 4 フィールドを持つ．保存する要約の合計は 8,192 文字，各フィールド合計 32 項目，1 項目 2,048 文字以下とする．TDO では S_1 と D_2 の合計に上限を適用する．構造検査の後，次の actor 入力全体を native tokenizer で計数し，従来の表示より短い場合だけ採用する．指示・課題・tools・予算通知・残存履歴を含めた判定であり，差分だけが短ければよいわけではない．共有の残予算と文脈長も確認する．

不採用または未送信では旧表示を維持する．既知の使用量は不採用 focus も計上し，使用量が不明になればその条件を停止する．入力帰属のための追加計数 [6] は両方式に共通で，その時間も作業上限に含む．

3. 実 験 方 法

3.1 課題と実行条件

AppWorld [7] の既知 4 課題（写真操作，精算，返金，CSV 処理）に対し，TR と TDO を各 1 回，計 8 条件実行した．model seed の基点は 157，world seed は 42 とし，役割ごとの呼出し数で model seed を増分する．課題を上記の順とし，奇数課題では TR から，偶数課題では TDO から実行する．各条件で新しい課題状態を用意し，両方式の初回入力が一致することを検査する．

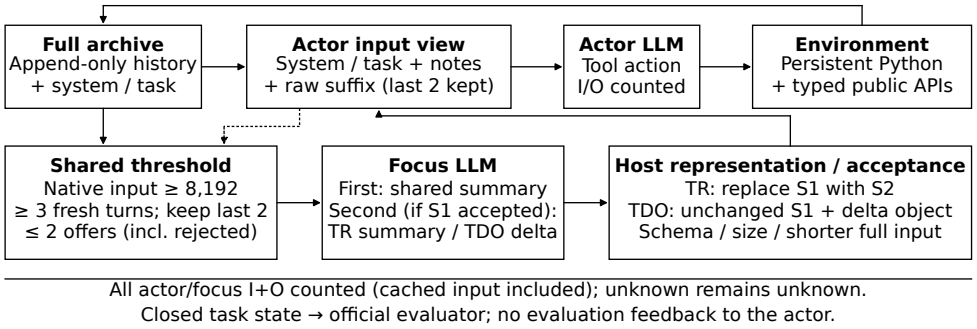


図 1　共通の入力長閾値で保持表現を比較するハーネス．
Fig. 1　Harness comparing retained representations under a common input-length threshold.

先行する同じ 4 課題・seed の 16,384 閾値比較では，両方式が 3/4 成功，費用 572,405 トークンで同値となり，第 2 差分処置は全課題で未発動だった．そこで本比較では，発動範囲を調べるため共通閾値 8,192 を結果観測後に選定した．作業上限も 1,800 秒から 1,200 秒へ変更するため，先行比較との費用差を閾値だけの効果とは扱わず，今回の同条件 TR/TDO 対を評価する．

actor と focus には MiniMax-M3（GGUF，IQ4_XS）を用いる．llama.cpp b11669，文脈長 65,536，KV cache q8_0，並列数 1，温度は actor 1.0，focus 0.7 とする．計算機は Apple M3 Ultra，メモリ 256 GiB である．各条件の上限は全モデル要求 80 回，論理入出力 450,000 トークン，作業 1,200 秒，1 回の通常出力 4,096 トークンとする．環境 API は写真操作 160 要求，他の課題 80 要求を上限とする．セットアップ，後片付け，評価には別に有限の上限を設ける．実装と全条件を公開する(注1)．

3.2　費用と品質

費用は，条件 p の actor と focus を含む全呼出し集合 C_p について，実入力 I_c と実出力 O_c を足した

$$T_p = \sum_{c \in C_p} (I_c + O_c) \tag{2}$$

とする．cached input を控除せず，失敗時や不採用要約の既知費用も含める．したがって，キャッシュを考慮した金銭費用とは区別する [5]．要約役を含む費用評価は ACON [4] にもあり，本稿では保持表現の評価指標として用いる．使用量不明があれば既知小計と不明件数を示し，確定総費用や削減率は算出しない．

(注1)：https://github.com/sophiria1225/AI-

最終品質は，実行後の状態に対する公式評価器の成否で測る．評価の詳細は実行中のモデルへ返さない．課題 t の成功を $y_{t,p} \in \{0, 1\}$ とし，TR の成功を TDO が失った課題数を

$$L = \sum_{t=1}^{4} \mathbf{1}[y_{t,\mathrm{TR}} = 1,\ y_{t,\mathrm{TDO}} = 0] \tag{3}$$

とする．成功数の総計だけでなく，課題ごとの保持を確認する．費用は全 4 対と，両方式が成功した課題集合 $\mathcal{S}$ の双方で比較する．未評価を失敗へ，使用量不明を 0 へ置き換えない．

固定した効用条件は，全 8 条件の評価・会計が揃い，$L = 0$，$\mathcal{S}$ が空でなく，全 4 対と共通成功対の両方で TDO の費用が小さいことである．処置への到達は，第 2object focus の既知送信と，その差分を含む actor 入力で確認する．処置前の履歴の対応も検査し，介入を伴う開発判定と集計上の効用条件を分けて報告する．

4.　結果と考察

4.1　課題別の成否と費用

表 1 と図 2 に全 8 条件を示す．

4.2　品質と全費用の比較

全 8 条件を評価し，成功は TR が 3/4，TDO が 2/4 だった．写真操作で TR の成功を TDO が失い，$L = 1$ となった．TDO の同課題はモデル呼出し中の時間上限で停止し，actor 1 呼出しの使用量が不明だった．既知小計は TR が 645,609，TDO が 546,325 トークンであるが，この差を総費用削減とは扱わない．固定した全 8 条件の会計充足条件を満たさないため，全体と共通成功集合の確定費用差・削減率はいずれも算出せず，効用条件と開発条件は不成立だった．

共通成功は精算と返金の 2 課題である．個別の確定

表 1　課題別の成否と論理入出力．不明呼出しは 0 補完しない．差分付き actor は既知使用量の呼出し数．

Table 1　Task outcomes and logical input/output. Known subtotals are distinguished from unknown usage.

課題	方式	最終成否	既知 token 小計	不明呼出し	差分付き actor
写真操作	TR	成功	200,016	0	0
写真操作	TDO	失敗	149,194	1	0
精算	TR	成功	182,483	0	0
精算	TDO	成功	218,061	0	13
返金	TR	成功	68,923	0	0
返金	TDO	成功	68,923	0	0
CSV 処理	TR	失敗	194,187	0	0
CSV 処理	TDO	失敗	110,147	0	6

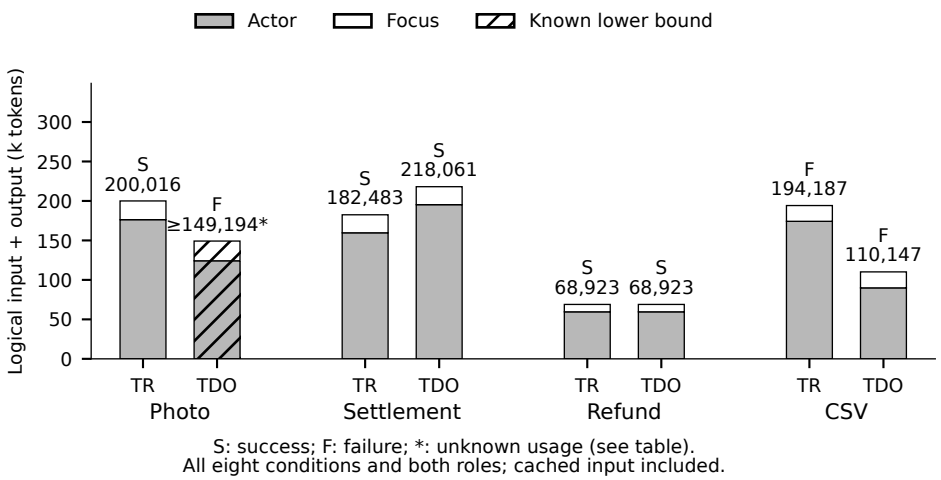


図 2　全 8 条件の actor・focus 別論理入出力と最終成否．斜線は確定総費用ではない既知小計を示す．

Fig. 2　Logical input/output by role and final outcomes. Hatched bars denote known lower bounds.

値を見ると，精算は TDO が 35,578 トークン多く，返金は同値だった．CSV 処理では TDO が 84,040 トークン少なかったが，両方式とも API 上限で停止し失敗した．この減少は，成功を保った節約の事例ではない．

4.3　差分処置の作動範囲

TDO の第 2object focus は写真操作，精算，CSV 処理の 3 課題で既知の送信を確認した．差分が採用されて actor 入力に届いたのは精算と CSV 処理の 2/4 課題で，差分付きの既知 actor 呼出しはそれぞれ 13 回，6 回だった．返金では第 2 差分処置が作動しておらず，その成功を差分保持の効果例には数えない．写真操作でも差分付き actor は 0 回であり，旧要約保持が成功喪失の原因だとまでは特定できない．

差分が届いた唯一の共通成功課題である精算では，focus 費用は 52 トークン減った一方，actor 費用は 35,630 トークン増えた．旧要約の再出力を要求しない処置が，後続入力や行動を含む総費用の減少に直結しない実例である．ただし，増分を旧要約の保持量だけに帰属する比較ではなく，処置後の履歴と呼出し数の変化を含む．以上より，本条件では機構の作動は観測できたが，品質を保つ全仕事費用削減は確認できなかった．

本比較は既知 4 課題，単一モデル・量子化・seed と結果観測後に選んだ閾値による開発評価である．seed は過去の別比較でも用いており，未見課題への独立確認ではない．結果は保持・出力指示・ホスト結合を合わせた処置の比較であり，各要素の単独効果を分離しない．同じ閾値でも処置後の履歴，呼出し回数，実時間は変わるため，課題ごとの成否と全呼出しの費用から解釈する．

5.　む　す　び

共通の入力長閾値で，再帰置換と旧要約保持・object

差分・ホスト結合を比較した．差分は 2 課題の actor 入力に届いたが，成功は 3/4 から 2/4 へ減った．差分が届いた共通成功課題では総費用も増えた．使用量不明 1 呼出しを残すため全体の削減率は確定せず，品質を保つ全仕事費用削減は確認できなかった．旧要約の再出力を省く局所的な処置は，その後の課題実行を含む効用を保証しない．

文　献

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” Proc. ICLR, May 2023.

[2] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” Proc. ICML, PMLR, vol.235, pp.50208–50232, July 2024.

[3] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, “LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression,” Proc. ACL, vol.1, pp.1658–1677, Aug. 2024. doi: 10.18653/v1/2024.acl-long.91.

[4] M. Kang, W.-N. Chen, D. Han, H. A. Inan, L. Wutschitz, Y. Chen, R. Sim, and S. Rajmohan, “ACON: Optimizing context compression for long-horizon LLM agents,” Proc. ICML, PMLR, vol.306, pp.55321–55375, July 2026.

[5] B. Xu, Z. Xue, D. Chen, C. Fu, C. Wu, C. Huang, C. Jiang, J. Fang, X. Deng, Y. Chen, Y. Yao, X. Wang, J. Shang, H. Chen, and N. Zhang, “TokenPilot: Cache-efficient context management for LLM agents,” Findings of ACL: EMNLP 2026, pp.19034–19049, Oct. 2026.

[6] V. K. Singh, P. Priyam, and G. Bhowmick, “Total cost of agency: Exact attribution of memory injection cost in multi-agent LLM workflows,” arXiv preprint arXiv:2609.23790v1, Sept. 2026.

[7] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian, “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” Proc. ACL, vol.1, pp.16022–16076, Aug. 2024. doi: 10.18653/v1/2024.acl-long.850.

（xxxx 年 xx 月 xx 日受付）

Abstract With a common threshold, we compare recursive replacement with an immutable-summary, object-delta bundle on four known tasks. Success falls from three to two tasks. One call has unknown usage; an activated common–success task costs more. The experiment does not establish quality-preserving whole-job savings.

Key words LLM agents, context compression, delta summarization, threshold control, cost evaluation